Ecossistema Assíncrono com Flask, PostgreSQL e ActiveMQ

1. O Cenário: Sistema de Logística "Expresso Real-Time"

A atividade consiste no desenvolvimento de uma aplicação de logística. O usuário acessa uma interface web e solicita a coleta de um pacote. Em uma arquitetura comum, o usuário ficaria aguardando a roteirização do caminhão para receber a confirmação.

Neste laboratório, implementaremos o **Desacoplamento**:

1. **Frontend**: Interface simples que envia o pedido e consulta o status periodicamente.
2. **Backend (Flask)**: Recebe a requisição, salva no **PostgreSQL** com o status PENDENTE e posta um evento na fila do **ActiveMQ**. Responde imediatamente ao usuário.
3. **Worker (Consumidor)**: Um script Python rodando em background que "escuta" a fila, simula o tempo de roteirização pesada (5 segundos) e atualiza o banco de dados para ENVIADO.

---

2. Passo a Passo da Implementação

Passo 1: Subindo a Infraestrutura (Docker Compose)

Para rodarmos o banco de dados e o broker sem poluir a máquina hospedeira, utilizaremos o Docker.

Crie o arquivo docker-compose.yml:

YAML

```yaml
version: '3.8'
services:
  activemq:
    image: rmohr/activemq:latest
    ports:
      - "8161:8161"   # Painel Administrativo Web
      - "61613:61613" # Porta do protocolo STOMP (usado pelo Python)
```

```yaml
  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin
      POSTGRES_DB: logistica
    ports:
      - "5432:5432"
```

Ação: Execute docker-compose up -d no terminal.

Passo 2: Preparando o Ambiente Python

Precisamos instalar os pacotes do microframework Flask, o driver do PostgreSQL e a biblioteca STOMP para a mensageria.

Ação: Execute no terminal:

Bash

```bash
pip install flask psycopg2-binary stomp.py
```

Passo 3: O Backend Produtor (app.py)

Este arquivo será a nossa API REST. Ele cria a tabela no banco automaticamente, recebe o POST do frontend, insere no banco e avisa o broker.

Python

```python
from flask import Flask, request, jsonify, render_template
import psycopg2
import stomp
import json

app = Flask(__name__)
```

```python
# Configuração do PostgreSQL
DB_CONN = "dbname='logistica' user='admin' host='localhost' password='admin'"

def init_db():
    conn = psycopg2.connect(DB_CONN)
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS pedidos (
            id SERIAL PRIMARY KEY,
            cliente VARCHAR(100),
            destino VARCHAR(100),
            status VARCHAR(20)
        )
    """)
    conn.commit()
    cur.close()
    conn.close()

# Configuração do Produtor ActiveMQ (STOMP)
def enviar_para_broker(pedido_id):
    conn = stomp.Connection([('localhost', 61613)])
    conn.connect('admin', 'admin', wait=True)

    mensagem = json.dumps({"pedido_id": pedido_id})
    # Envia a mensagem para a fila 'logistica.entregas'
    conn.send(body=mensagem, destination='/queue/logistica.entregas')
    conn.disconnect()

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/api/pedidos', methods=['POST'])
def criar_pedido():
    dados = request.json
    cliente = dados.get('cliente')
    destino = dados.get('destino')

    # 1. Salva no Postgres com status PENDENTE (Síncrono)
    conn = psycopg2.connect(DB_CONN)
    cur = conn.cursor()
    cur.execute(
```

```
        "INSERT INTO pedidos (cliente, destino, status) VALUES (%s, %s, %s) RETURNING id",
        (cliente, destino, 'PENDENTE')
    )
    pedido_id = cur.fetchone()[0]
    conn.commit()
    cur.close()
    conn.close()

    # 2. Emite o evento para o ActiveMQ (Assíncrono)
    enviar_para_broker(pedido_id)

    # 3. Libera a tela do usuário imediatamente
    return jsonify({"mensagem": "Pedido recebido!", "id": pedido_id}), 201

@app.route('/api/pedidos', methods=['GET'])
def listar_pedidos():
    conn = psycopg2.connect(DB_CONN)
    cur = conn.cursor()
    cur.execute("SELECT id, cliente, destino, status FROM pedidos ORDER BY id DESC")
    pedidos = [{"id": linha[0], "cliente": linha[1], "destino": linha[2], "status": linha[3]} for linha in
cur.fetchall()]
    cur.close()
    conn.close()
    return jsonify(pedidos)

if __name__ == '__main__':
    init_db()
    app.run(debug=True, port=5000)
```

Passo 4: O Consumidor em Background (worker.py)

Este script rodará em um terminal separado. Ele não tem interface gráfica; seu único trabalho é ouvir o ActiveMQ e processar os dados pesados.

Python

```
import time
import stomp
```

```python
import json
import psycopg2

DB_CONN = "dbname='logistica' user='admin' host='localhost' password='admin'"

class LogisticaListener(stomp.ConnectionListener):
    def on_message(self, frame):
        dados = json.loads(frame.body)
        pedido_id = dados['pedido_id']

        print(f"[WORKER] Recebeu evento para roteirizar o pedido ID: {pedido_id}")

        # Simula um processamento pesado (ex: cálculo de rotas no Google Maps)
        time.sleep(5)

        # Atualiza o status no PostgreSQL
        conn = psycopg2.connect(DB_CONN)
        cur = conn.cursor()
        cur.execute("UPDATE pedidos SET status = 'ENVIADO' WHERE id = %s", (pedido_id,))
        conn.commit()
        cur.close()
        conn.close()

        print(f"[WORKER] Pedido {pedido_id} processado com sucesso!")

# Conecta ao ActiveMQ
conn = stomp.Connection([('localhost', 61613)])
conn.set_listener('', LogisticaListener())
conn.connect('admin', 'admin', wait=True)

# Inscreve-se na fila
conn.subscribe(destination='/queue/logistica.entregas', id=1, ack='auto')

print("[WORKER] Aguardando novos pedidos...")
while True:
    time.sleep(1) # Mantém o script rodando
```

Passo 5: O Frontend (templates/index.html)

Crie uma pasta chamada templates na mesma raiz do app.py e insira este arquivo. Ele fará o consumo da API via fetch do JavaScript e fará o polling (atualização) da lista a cada 2 segundos para o aluno ver a mágica do processamento assíncrono acontecendo na tela.

HTML

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Expresso Logística</title>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f4f4f9; padding: 20px; }
    .container { max-width: 600px; margin: auto; background: white; padding: 20px; border-radius: 8px;
box-shadow: 0 0 10px rgba(0,0,0,0.1); }
    input, button { padding: 10px; margin: 5px 0; width: 100%; box-sizing: border-box; }
    button { background-color: #28a745; color: white; border: none; cursor: pointer; }
    .pedido { padding: 10px; border-bottom: 1px solid #ddd; display: flex; justify-content: space-
between; }
    .status-PENDENTE { color: orange; font-weight: bold; }
    .status-ENVIADO { color: green; font-weight: bold; }
  </style>
</head>
<body>
  <div class="container">
    <h2>Solicitar Entrega</h2>
    <input type="text" id="cliente" placeholder="Nome do Cliente">
    <input type="text" id="destino" placeholder="Endereço de Destino">
    <button onclick="enviarPedido()">Despachar Pedido</button>

    <h3>Pedidos Recentes</h3>
    <div id="lista-pedidos"></div>
  </div>

  <script>
    async function enviarPedido() {
      const cliente = document.getElementById('cliente').value;
      const destino = document.getElementById('destino').value;

      await fetch('/api/pedidos', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ cliente, destino })
      });
```

```
        document.getElementById('cliente').value = '';
        document.getElementById('destino').value = '';
        carregarPedidos();
    }

    async function carregarPedidos() {
        const response = await fetch('/api/pedidos');
        const pedidos = await response.json();
        const lista = document.getElementById('lista-pedidos');
        lista.innerHTML = '';

        pedidos.forEach(p => {
            lista.innerHTML += `
                <div class="pedido">
                    <span>#${p.id} - ${p.cliente} (${p.destino})</span>
                    <span class="status-${p.status}">${p.status}</span>
                </div>
            `;
        });
    }

    // Atualiza a lista a cada 2 segundos para ver a mudança assíncrona
    setInterval(carregarPedidos, 2000);
    carregarPedidos();
  </script>
</body>
</html>
```

---

3. Guia de Teste e Execução (O que o aluno deve observar)

Para validar a compreensão dos conceitos, a execução deve seguir esta ordem exata:

1. **Gatilho Inicial**: Com o Docker rodando, abra dois terminais diferentes.
2. **Inicie o Backend**: No Terminal 1, rode python app.py. Acesse http://localhost:5000.
3. **O Comportamento Sem Consumidor**: Não inicie o worker.py ainda. Cadastre um pedido na tela.
    - *Observação*: A tela do usuário não travou. O pedido aparecerá com status **PENDENTE**. Como o Worker está desligado, ele continuará pendente para sempre. O aluno deve ir em http://localhost:8161 (ActiveMQ) e ver que há **1 mensagem**

enfileirada aguardando. Isso demonstra a **Persistência e Tolerância a Falhas**.

4. **A Mágica do Desacoplamento**: No Terminal 2, rode python worker.py.
    - *Observação*: O Worker ligará e imediatamente "puxará" a mensagem atrasada da fila, consumindo-a.
5. **Fluxo Completo**: Volte ao navegador e cadastre um novo pedido.
    - *Observação*: O aluno verá o status nascer como **PENDENTE**. Passados exatos 5 segundos (o time.sleep do Worker), o JavaScript no frontend atualizará a tela automaticamente para **ENVIADO**.